

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/224645204>

Applying Code Analysis and 3D Design Pattern Grouping to Facilitate Program Comprehension

Conference Paper · January 2005

DOI: 10.1109/VISOF.2005.1684320 · Source: IEEE Xplore

CITATIONS

2

READS

37

2 authors, including:



[Juergen Rilling](#)

Concordia University

128 PUBLICATIONS 2,564 CITATIONS

SEE PROFILE

Applying Code Analysis and 3D Design Pattern Grouping to Facilitate Program Comprehension

Juergen Rilling and Vu-Loc Nguyen
Department of Computer Science and Software Engineering
Concordia University
Montreal, QC, Canada

ABSTRACT

The increasing size and complexity of software systems introduces new challenges in comprehending the overall structure of programs. Modeling languages and notations were introduced to provide abstractions from existing source code during forward engineering. However, these same modeling techniques and notations fail during source code reverse engineering due to: (1) Information overload, and (2) the existence of a conceptual gap between the abstractions derived during forward and reverse engineered. Our tool uses a 3D representation for UML in combination with source code analysis to facilitate the comprehension process. We also address issues of crosscutting, navigation, and the use of animation to visualize design patterns.

Keywords

3D visualization, program comprehension, design patterns

1. 3D visualization of design patterns

Combining source code analysis techniques with more sophisticated visualization techniques can assist users in comprehending data and identifying patterns and relationships that would not otherwise be obvious. In our tool, we present a 3D UML visualization in combination with design pattern recovery techniques that provide additional insights during program comprehension. In particular, we focus on closing the conceptual gap between reverse engineering and forward engineering.

1.1 Visualizing design pattern

Traditional visualization tools are limited in their support for visual representation and recognition of design patterns. These limitations are caused by several factors including lack of code analysis techniques to recover design patterns from source code, the metaphor (typically UML) itself does not provide adequate support for visualizing design patterns and issues related to the layout algorithms possibly by focusing on edge crossing rather than grouping. There are several UML notation extensions to visually represent individual design pattern within an aggregate of patterns [1,3]. A Venn Diagram-Style pattern annotation can be introduced using shading to indicate participants of the pattern. Classes related to a particular design pattern are encapsulated within a transparent sphere. This approach works well for smaller numbers of patterns, and in cases where there are no crosscuttings

among design patterns. Shading also reduces the readability of the diagrams and does not identify the role of the participant classes in the pattern. In our tool, we provide a separate view showing only the classes participating in the design pattern and lay them out as presented by the “Gang of Four” [2] by locally optimizing the layout to conform to the design pattern specific layouts. The layout algorithm uses information regarding the different roles of the classes derived by our design pattern recovery algorithm to reorganize the layout. Currently, we support 10 of the GoF design patterns.

The third dimension makes it possible to “stack” more information within the given screen space [4]. For example, classes that share a role can be made visible without destroying the original design pattern layout. Viewed from the “front”, the layout is easily recognizable as a “bridge” pattern. (It should be noted that the notion of “front” has less of a meaning in 3D unless we orient ourselves accordingly). The familiarity of the design pattern layout will make the design pattern easier to recognize amongst other classes. Figure 1 shows a *Bridge* pattern, where all classes belonging to the pattern are grouped and laid out according to the layout presented by the GoF [2]. Classes that share the same role within a design pattern can be stacked “behind” each other. Classes related to the design pattern are encapsulated within a transparent sphere, and classes that participate in more than one pattern will be moved to the spheres intersection.

1.2 Crosscutting among design patterns

One major problem with grouping classes within their respective design patterns is that some classes participate in more than one design pattern (also known as crosscutting). For example, one class can play the role of a *concrete product* in an *abstract factory* pattern and also participate as a concrete implementor in another *bridge* pattern. The challenge is to determine where to physically position the class participating in multiple patterns on the screen. Should it be grouped with the abstract factory pattern or should it be grouped with the bridge pattern? Assigning the class to one pattern will leave the other patterns and their layout incomplete. 3D supports the notion of depth, animation, and the ability to follow gradual animated changes within multiple camera views.

Our tool supports all of these aspects as well as the ability to provide different 3D viewing angles without the need to reorganize the diagrams.

“Compact mode view”. In this view, classes are generally grouped with their respective design patterns, as long as there is no crosscutting exists among patterns. The view might lead to some distortion in the design pattern layout which makes these patterns visually less identifiable.

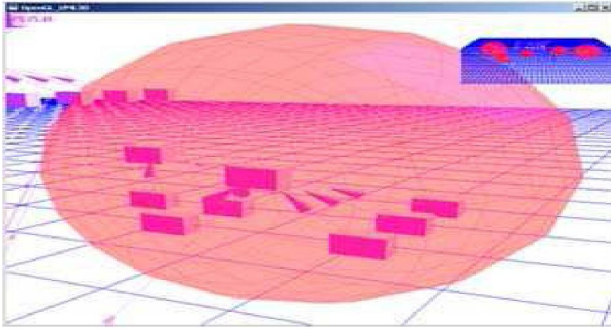


Figure 1 The “Bridge” pattern shown in 3D

“Relaxed mode view”. In this view, patterns are laid out in the GoF style, even in situations where crosscutting occurs. Classes which participate in more than one pattern will appear multiple times in the diagram. (Figure 2).

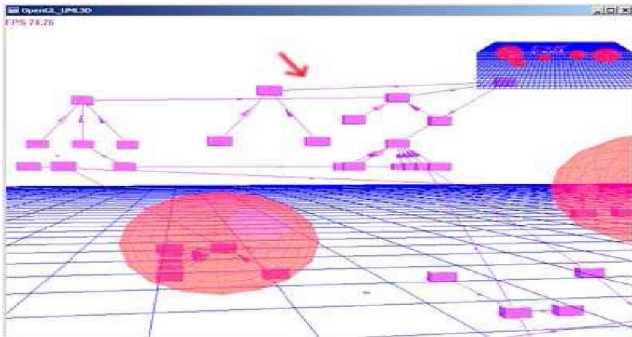


Figure 2. “Relaxed mode view”

A drawback of this approach is an increase in the diagram complexity due to the larger number of nodes to be displayed. We address this issue by supporting filtering techniques that can be applied either on individual design patterns or pattern types. Transition among the views is animated to reduce user disorientation by providing a visual link between the two views. The animation focuses on the transition where classes that participate in several patterns are duplicated. A wire-frame copy denoting the migration endpoint indicates the transition of each “class split”. Users can focus the camera on the migration of a particular class its migration path.

Filtering support. We currently support filtering of design patterns based on pattern type as well as pattern name. The patterns in focus are displayed in the center of the screen, and all other detected patterns are encapsulated in red transparent spheres. Filtering by pattern type allows users

to choose between visualizing all occurrences of a particular pattern type or a particular pattern instance. For the latter approach, one can browse through all identified patterns or search for a particular name/type combination.

2. Discussion and Conclusions

By adopting 3D techniques that are commonly used in other domains, like the game industry or information visualization, 3D UML can become an important extension to the traditional 2D UML representation [6]. The benefits of 3D visualization depend on the content to be displayed, the metaphor used, and the availability of adequate analysis, filtering and browsing techniques. In particular, without analysis and filtering support, 3D visualization will face the same challenges as traditional 2D visualization approaches [3]. Our pattern visualization tool is based on a general grid layout algorithm to provide the initial system layout. In a second step, the layout is locally optimized by a second design pattern specific layout algorithm. In cases with a relative large number of crosscuttings and using the compact view, the pattern layout algorithm will not be able to compute a local optimum (GoF design pattern layout). In the relaxed view mode, patterns can be “moved out” from the original grid layout position to become more visible. The major advantage of the 3D approach is the ability to stack classes with the same role, and the option to relax the notion of line crossing for the grid layout, since one can use a 3D grid layout. The effectiveness of the 3D visualization depends on several factors: (1) Grouping and layout depend on the availability of source code analysis techniques; (2) Scalability continues to be an important issue, which requires additional filtering and abstraction techniques; and (3) The UML metaphor lacks the expressiveness to represent semantic grouping/information like design patterns.

3. References

- [1] Jing Dong and Sheng Yang, “Visualizing Design Patterns With A UML Profile”, In the Proc. of the IEEE Symposium on VL, Auckland, New Zealand, Oct. 2003, pp. 123-125.
- [2] E. Gamma, R. Helm, R. Johnson, J. Vlissides “Design Patterns – Elements of Reusable Object-Oriented Software”, Addison-Wesley, 1994.
- [3] Maletic, J.I., Leigh, J., Marcus, A., Dunlap, G., “Visualizing Object-Oriented Software in Virtual Reality, *IEEE IWPC 2001*”, 2001, pp. 26-35.
- [4] Nielsen, J., “2D is Better Than 3D”, *AlertBox*, <http://useit.com/alertbox/981115.html>, 1998.
- [5] Rilling J. and Mudur S. P., “On the Use of Metaballs to Visually Map Source Code Structures and Analysis Results onto 3D Space”, *Proc. IEEE WRCE 2002*, pp. 132-142.
- [6] Shneiderman, B., “Tree Visualization with Tree-Maps: A 2-D Space-Filling Approach”. *ACM Trans. of CHI*, Vol. 11, No.1, 1992, pp. 92-99.